

§ identity *which PPM, which name, which version of which standard* 8 fields · 8 leaves

model **name** **api_group** **since** **deprecated_in** **desc**
standard_refs[] **type_family[]** *the type suffixes this entry is generic over*

§ bindings *how the call is spelled per language; null = not offered* 5 fields · 15 leaves

c, expanded — the other four are similar
expressible **header** **signature** **fortran90 5** **fortran08 4**
lis 1 **cpp 2**

§ execution *when and where the call runs; 8 of 11 closed enums* 11 fields · 11 leaves

blocking *blocks until locally complete*
execute_once *lifetime singleton, e.g. MPI_Init*
collective *every process in the team calls it*
thread_safety *single, ..., multiple — minimum level the call needs*
launch *cpu, gpu, cpu+gpu — where in code it may be issued*
runs_on *cpu, gpu, cpu+gpu — what hardware does the work*
gpu_scope *thread, warp, block, device — who calls together*
stages[] *init, start, complete, free — MPI-4 taxonomy*
locality *local, remote, both — whose state the call touches*
completion *incomplete, completing, freeing, stream_ordered*
procedure_class *blocking, nonblocking, persistent (+1)*

§ semantics *what it means; exactly one of the first four is non-null* 6 fields · 65 leaves

collective, expanded — the other three operation classes are equally small
type **root_involved** **in_place** **comm_scope** **algorithm_class**
reduction_op **topology_aware** *in_place = documented in-place mode*

point_to_point 4 *matched send and receive pairs*
one_sided 3 *put, get, accumulate — RMA*
atomic 3 *atomic read-modify-write*

memory 6 *when buffers may be reused*

formal, expanded — null for query and management calls, a real answer

participants[] 5 *who is involved, and does each one call*
id **role_kind** **calls_function** **cardinality** **selector**
data_flow[] 10 *one entry per memory access, with its provenance*
id **participant** **op** **buffer** **extent 2** **offset** **inputs[] 2**
operation_ref **valid_during[] 3** **source 6** *one of 4 kinds*
sync 3 *events, ordering, cross-call-instance matching*
events[] 6 **ordering[] 4** **matching 4**
handle_lifecycle[] 4 *a handle across create, use and destroy*
handle **handle_kind** **role** **derived_from**

§ parameters [] *per argument: role, direction, type, constraints* 15 fields · 21 leaves

kind *the argument's role — BUFFER, COUNT, RANK and 28 more*
direction *in, out, inout — relative to the caller*
asynchronous *runtime may touch it after return*
length *a count, or the sibling parameter holding it*
constraints 3 **not_null** **value_range** **paired_with**
memory_space *host, device, symmetric_heap (+2) — where it lives*
name **desc** **binding_type 5** **constant** **pointer** **array_type**
func_type **parameter_bindings[]** **root_only**

§ return *what the return value means — the C type does not settle it* 5 fields · 9 leaves

kind *ERROR_CODE, RESULT, ... — a status, an object, or a datum*
value_kind **binding_type 5** **possible_errors[]**
possible_successes[]

§ tool_integration *hooks and preconditions for consuming tools* 4 fields · 7 leaves

profiling_name *interposer symbol, e.g. PMPI_**
invariants[] *preconditions in the shared grammar*
buffer_aliasing *forbidden, allowed, in_place_only*
context_dependencies[] 4
field **default_value** **depends_on[]** **note**

§ relationships *edges to other entries (model:function_key)* 3 fields · 8 leaves

variants 6 **superseded_by** **supersedes[]**

 closed enum  boolean  free text  nested object 144 leaves
 entry reference  n fields  array  exactly one of 41 enums
230 values